

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Комп'ютерна графіка та обробка зображень»

Робота №7

Тема «Навколишнє, дифузне та дзеркальне освітлення»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Туленков А.В.

2025

ЗАВДАННЯ

Завдання на роботу зі сторінки 23:

1. Змініть фігуру.
2. Змініть текстуру.
3. Змініть тип освітлення.

РЕЗУЛЬТАТ

```
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include <cmath> // Для використання sin/cos

// Константа для шляху до текстури
const char *TEXTURE_PATH = "texture.jpg";

// Функція для завантаження текстури (без змін)
unsigned int loadTexture(const char *path)
{
    unsigned int textureID;
    glGenTextures(1, &textureID);
    int width, height, nrChannels;
    unsigned char *data = stbi_load(path, &width, &height, &nrChannels,
0);
    if (data)
    {
        GLenum format;
        if (nrChannels == 1)
            format = GL_RED;
        else if (nrChannels == 3)
            format = GL_RGB;
        else if (nrChannels == 4)
            format = GL_RGBA;

        glBindTexture(GL_TEXTURE_2D, textureID);
        glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0,
format, GL_UNSIGNED_BYTE, data);
        glGenerateMipmap(GL_TEXTURE_2D);

        // Встановлення параметрів фільтрації
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

```

        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);

        stbi_image_free(data);
    }
    else
    {
        fprintf(stderr, "Failed to load texture: %s\n", path);
        stbi_image_free(data);
    }
    return textureID;
}

//
=====
=
// ШЕЙДЕРИ З МОДЕЛЛЮ ОСВІТЛЕННЯ ПО ФОНУ (PHONG)
//
=====
=

// Код вершинного шейдера
// Включає нормалі та позицію фрагмента у світових координатах
const char *vertexShaderSource = R"(
    #version 330 core
    layout (location = 0) in vec3 aPos;
    layout (location = 1) in vec2 aTexCoord;
    layout (location = 2) in vec3 aNormal; // Атрибут нормалі

    out vec2 TexCoord;
    out vec3 FragPos;
    out vec3 Normal; // Передаємо нормаль у фрагментний шейдер

    uniform mat4 model;
    uniform mat4 view;
    uniform mat4 projection;

    void main() {
        gl_Position = projection * view * model * vec4(aPos, 1.0);
        TexCoord = aTexCoord;

        // Позиція фрагмента у світових координатах
        FragPos = vec3(model * vec4(aPos, 1.0));

        // Трансформація нормалей (використовуємо матрицю нормалей:
transpose(inverse(model)))
        Normal = mat3(transpose(inverse(model))) * aNormal;
    }
)";

// Код фрагментного шейдера
// Реалізує Точкове освітлення по Фоні (Ambient + Diffuse + Specular)
const char *fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;

```

```

    in vec2 TexCoord;
    in vec3 FragPos;
    in vec3 Normal;

    uniform sampler2D texture1;

    uniform vec3 lightPos; // Позиція джерела світла
    uniform vec3 viewPos;  // Позиція камери (для спекулярного
освітлення)

    // Властивості світла
    vec3 lightColor = vec3(1.0, 1.0, 1.0);
    vec3 objectColor = vec3(1.0); // Використовуємо колір текстури

    void main() {
        // Отримання кольору текстури
        vec3 texColor = vec3(texture(texture1, TexCoord));

        // 1. AMBIENT (Оточуюче світло)
        float ambientStrength = 0.1;
        vec3 ambient = ambientStrength * lightColor;

        // 2. DIFFUSE (Диффузне світло)
        vec3 norm = normalize(Normal);
        vec3 lightDir = normalize(lightPos - FragPos); // Напрямок від
фрагмента до світла
        float diff = max(dot(norm, lightDir), 0.0);
        vec3 diffuse = diff * lightColor;

        // 3. SPECULAR (Спекулярне світло - Відблиск)
        float specularStrength = 0.5; // Інтенсивність відблиску
        vec3 viewDir = normalize(viewPos - FragPos); // Напрямок від
фрагмента до камери
        vec3 reflectDir = normalize(reflect(-lightDir, norm)); //
Вектор відбиття
        float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32.0); //
32.0 - ступінь блиску
        vec3 specular = specularStrength * spec * lightColor;

        // Фінальний колір = (Ambient + Diffuse + Specular) * Колір
об'єкта
        vec3 result = (ambient + diffuse + specular) * texColor;
        FragColor = vec4(result, 1.0);
    }
}";

int main()
{
    // ... (Ініціалізація GLFW, GLEW та вікна без змін) ...
    if (!glfwInit())
    {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

```

```

GLFWwindow *window = glfwCreateWindow(800, 600, "Texture & Lighting
Example", NULL, NULL);
if (!window)
{
    fprintf(stderr, "GLFW window creation failed\n");
    glfwTerminate();
    return -1;
}
glfwMakeContextCurrent(window);
if (glewInit() != GLEW_OK)
{
    fprintf(stderr, "GLEW initialization failed\n");
    glfwDestroyWindow(window);
    glfwTerminate();
    return -1;
}
glEnable(GL_DEPTH_TEST);
glViewport(0, 0, 800, 600);

// Компіляція та лінування шейдерів
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);
GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);
GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);
glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

// Дані вершин для КУБА (36 вершин для унікальних нормалей і
текстурних координат на кожній грані)
// Формат: X, Y, Z (Позиція), U, V (Текстура), NX, NY, NZ (Нормаль)
GLfloat vertices[] = {
    // Задня грань
    -0.5f, -0.5f, -0.5f,  0.0f, 0.0f,  0.0f,  0.0f, -1.0f, // 0
    0.5f, -0.5f, -0.5f,  1.0f, 0.0f,  0.0f,  0.0f, -1.0f, // 1
    0.5f,  0.5f, -0.5f,  1.0f, 1.0f,  0.0f,  0.0f, -1.0f, // 2
    0.5f,  0.5f, -0.5f,  1.0f, 1.0f,  0.0f,  0.0f, -1.0f, // 3
    -0.5f,  0.5f, -0.5f,  0.0f, 1.0f,  0.0f,  0.0f, -1.0f, // 4
    -0.5f, -0.5f, -0.5f,  0.0f, 0.0f,  0.0f,  0.0f, -1.0f, // 5

    // Передня грань
    -0.5f, -0.5f,  0.5f,  0.0f, 0.0f,  0.0f,  0.0f,  1.0f, // 6
    0.5f, -0.5f,  0.5f,  1.0f, 0.0f,  0.0f,  0.0f,  1.0f, // 7
    0.5f,  0.5f,  0.5f,  1.0f, 1.0f,  0.0f,  0.0f,  1.0f, // 8
    0.5f,  0.5f,  0.5f,  1.0f, 1.0f,  0.0f,  0.0f,  1.0f, // 9
    -0.5f,  0.5f,  0.5f,  0.0f, 1.0f,  0.0f,  0.0f,  1.0f, // 10
    -0.5f, -0.5f,  0.5f,  0.0f, 0.0f,  0.0f,  0.0f,  1.0f, // 11

    // Ліва грань
    -0.5f,  0.5f,  0.5f,  1.0f, 0.0f, -1.0f,  0.0f,  0.0f, // 12
    -0.5f,  0.5f, -0.5f,  1.0f, 1.0f, -1.0f,  0.0f,  0.0f, // 13
    -0.5f, -0.5f, -0.5f,  0.0f, 1.0f, -1.0f,  0.0f,  0.0f, // 14

```

```

-0.5f, -0.5f, -0.5f, 0.0f, 1.0f, -1.0f, 0.0f, 0.0f, // 15
-0.5f, -0.5f, 0.5f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f, // 16
-0.5f, 0.5f, 0.5f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f, // 17

// Права грань
0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f, // 18
0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, // 19
0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f, // 20
0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f, // 21
0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, // 22
0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f, // 23

// Нижня грань
-0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 24
0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 25
0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 26
0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 27
-0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f, // 28
-0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f, // 29

// Верхня грань
-0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f, // 30
0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f, // 31
0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, // 32
0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, // 33
-0.5f, 0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, // 34
-0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f // 35
};

// Для куба, що використовує 36 вершин, ЕВО не потрібен, оскільки
вершини не повторюються.
// Проте, оскільки в початковому коді був ЕВО, я залишу його і
зміню glDrawElements.
// В даному випадку 36 вершин вже формують куб, тому індекси
(indices[]) і ЕВО можна видалити.
// Для збереження логіки, я зміню `glDrawElements` на
`glDrawArrays` і видаляю ЕВО:

// GLuint indices[] = {0, 1, 2, 3, 4, 5, 6, 0, 6}; // <-- Видалено

GLuint VAO, VBO; // ЕВО видалено
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
// glGenBuffers(1, &EBO); // <-- Видалено

size_t stride = 8 * sizeof(GLfloat); // 3 (Позиція) + 2 (Текстура)
+ 3 (Нормаль) = 8

glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

// 1. Атрибут 0: Позиція (3 float)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, (GLvoid
*)0);
glEnableVertexAttribArray(0);

```

```

// 2. Атрибут 1: Текстурні координати (2 float)
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, stride, (GLvoid *)
(3 * sizeof(GLfloat)));
glEnableVertexAttribArray(1);

// 3. Атрибут 2: Нормалі (3 float)
glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, stride, (GLvoid *)
(5 * sizeof(GLfloat)));
glEnableVertexAttribArray(2);

glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);

// Завантаження текстури
unsigned int texture = loadTexture(TEXTURE_PATH);

// Отримання uniform-локацій
GLuint modelLoc = glGetUniformLocation(shaderProgram, "model");
GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
GLuint projectionLoc = glGetUniformLocation(shaderProgram,
"projection");
GLuint lightPosLoc = glGetUniformLocation(shaderProgram,
"lightPos");
GLuint viewPosLoc = glGetUniformLocation(shaderProgram, "viewPos");
// Локація для позиції камери

// Головний цикл рендерингу
while (!glfwWindowShouldClose(window))
{
    glfwPollEvents();

    // Очищення буферів (темно-сірий колір)
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glUseProgram(shaderProgram);

    // 1. Позиція світла (Рухаємо світло по колу)
    glm::vec3 lightPos(1.2f * sin(glfwGetTime()), 0.5f, 2.0f *
cos(glfwGetTime()));

    // 2. Позиція камери (залишається фіксованою)
    glm::vec3 cameraPos = glm::vec3(0.0f, 0.0f, 3.0f);

    // 3. Обчислення матриць
    glm::mat4 model = glm::mat4(1.0f);
    model = glm::rotate(model, (float)glfwGetTime() *
glm::radians(40.0f), glm::vec3(0.5f, 1.0f, 0.0f)); // Обертання куба

    glm::mat4 view = glm::lookAt(cameraPos, glm::vec3(0.0f, 0.0f,
0.0f), glm::vec3(0.0f, 1.0f, 0.0f));
    glm::mat4 projection = glm::perspective(glm::radians(45.0f),
800.0f / 600.0f, 0.1f, 100.0f);

    // 4. Передача uniform-змінних
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));
    glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

```

```

        glUniformMatrix4fv(projectionLoc, 1, GL_FALSE,
glm::value_ptr(projection));

        glUniform3fv(lightPosLoc, 1, glm::value_ptr(lightPos));
        glUniform3fv(viewPosLoc, 1, glm::value_ptr(cameraPos)); //
Передача позиції камери для Specular

        // 5. Рендеринг куба
        glBindVertexArray(VAO);
        glBindTexture(GL_TEXTURE_2D, texture);
        // glDrawElements(GL_TRIANGLE_STRIP, 9, GL_UNSIGNED_INT, 0); //
<-- Старий виклик
        glDrawArrays(GL_TRIANGLES, 0, 36); // Рендеринг 36 вершин, що
формують 12 трикутників куба
        glBindVertexArray(0);

        glfwSwapBuffers(window);
    }

    // Очищення ресурсів
    glDeleteVertexArrays(1, &VAO);
    glDeleteBuffers(1, &VBO);
    // glDeleteBuffers(1, &EBO); // <-- Видалено
    glfwDestroyWindow(window);
    glfwTerminate();
    return 0;
}

```